Лабораторная работа № 4

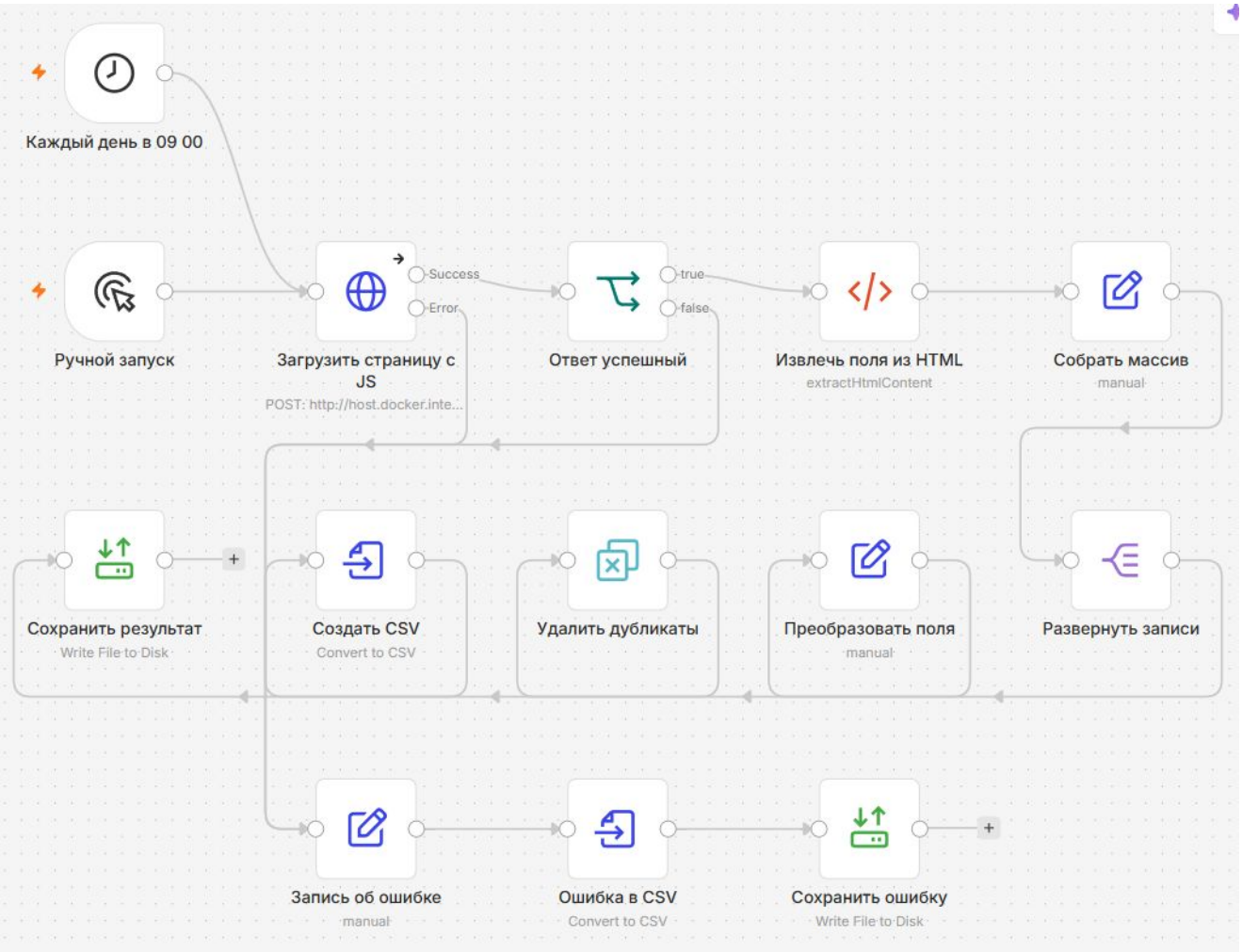
Сбор данных с динамических страниц

Вшивков К. И., ПМИ-3-2025. Вариант 6.

Цель: собрать данные через браузер и прямые HTTP-запросы, проверить качество и сравнить результат с n8n.

Получены 207 товаров: 147 из ScrapingCourse и 60 ноутбуков из Web Scraper. Оба способа Python дали одинаковые данные. В n8n собрана контрольная выборка из 22 товаров.

1. Схема n8n



Рендерер открывает страницу с JavaScript и нажимает Load more. Далее n8n извлекает поля, разворачивает массив, приводит цену к числу, удаляет дубли и записывает CSV. Для ошибки предусмотрена отдельная ветка.

Расписание - ежедневно в 09:00 по Перми. Автоматический запуск выключен до публикации workflow.

2. Диагностика источников

Проверка	ScrapingCourse	Web Scraper
Начальная страница	12 карточек в HTML	Карточки создаются JavaScript
Действие	Load more; GET /ajax/products?offset=10	More; добавляет 6 карточек
Данные без браузера	HTML в ответе AJAX, шаг offset = 10	JSON в атрибуте data-items
Без JavaScript	После клика остаются 12 карточек	Список не появляется
Выборка	Все 147 товаров	Первые 60 из 117 ноутбуков

В исходном HTML ScrapingCourse нет текста товара из следующей порции. С JavaScript один клик увеличивает число карточек до 24. Начальный HTML, ответы диагностики и снимки сохранены.

3. Реализация сбора

Playwright создаёт browser, context и page. Заданы User-Agent, окно 1280 × 900, локаль en-US, таймауты 15 и 30 секунд. Закрытие ресурсов находится в finally. После клика ожидаются AJAX-ответ и рост DOM; пауза 0,7 секунды ограничивает частоту запросов.

Предохранители: не более 30 итераций, лимиты 240 и 60 товаров, остановка после двух итераций без прироста. Поля извлекаются одним locator.evaluate_all. Повторы удаляются по product_id.

Локатор	Смена класса	Новый div	Перевод текста
get_by_test_id('load-more-btn')	Устойчив	Устойчив	Устойчив
#product-grid .product-item	Нужна правка	Устойчив	Устойчив
a[href] внутри карточки	Устойчив	Устойчив	Устойчив
[itemtype="https://schema.org/Product"]	Устойчив	Устойчив	Устойчив

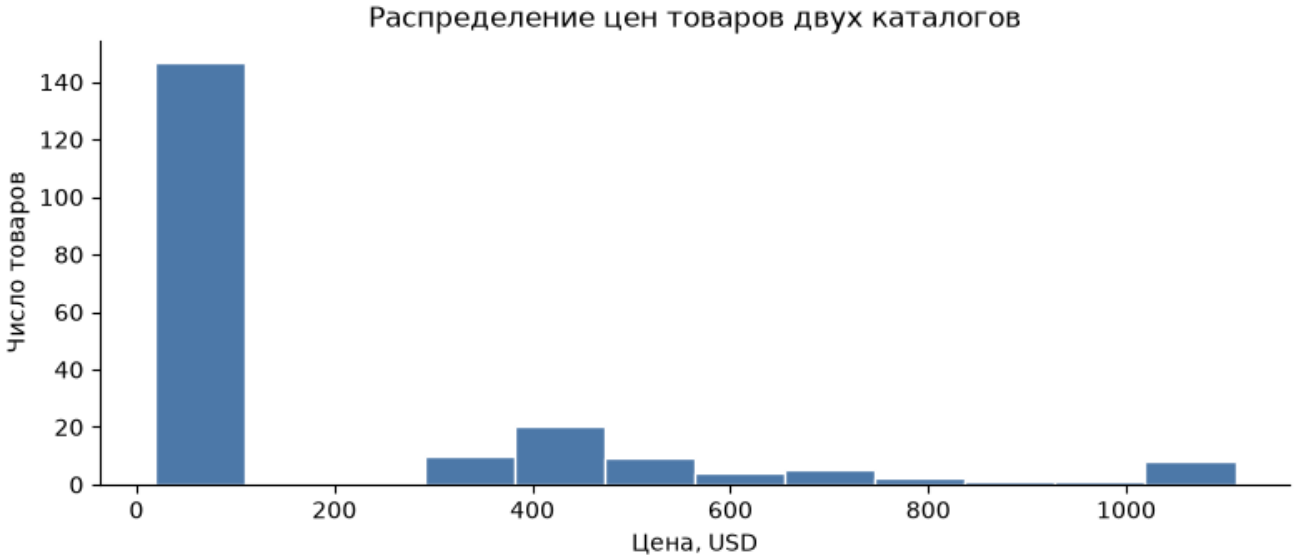
Резерв для карточек: ссылки #product-grid a[href*="/ecommerce/product/"]. В Web Scraper класс кнопки a.ecomerce-items-scroll-more нужно проверить после редизайна.

В лёгком варианте используется requests.Session. Для основного каталога цикл AJAX завершается на ответе 204 или пустой порции. Для второго каталога достаточно одного HTML-запроса и json.loads для data-items.

4. Результаты и качество

Показатель полного сбора	Playwright	requests
Уникальных товаров	207	207
Нажатий кнопки	24	Не требуются
Время, с	39,8591	16,5681
Запросов	60	17
Тела ответов, МБ	4,552773	0,208613
Пиковая память, МБ	585,474	108,306

Поля: product_id, name, price, currency, product_url. Обязательные поля заполнены, цены положительные, дубликатов нет. Таблицы двух способов совпали; контрольные 22 товара n8n совпали с соответствующей частью полного набора.



5. Обработка сбоев

Smoke-check обнаружил более 5% пустых обязательных полей в специально испорченной копии. Основной набор не менялся. В diagnose.py ожидался отсутствующий элемент: TimeoutError перехвачен, HTML и PNG сохранены, дальнейшее чтение карточек выполнено.

Журнал collection_log.csv содержит запуск, итерацию, число карточек, прирост, время, статус и URL. Ошибочный запуск сборщика завершается ненулевым кодом. Сохранены robots.txt источников; используются паузы и лимиты.

6. Сравнение способов

Контрольный замер от 25.10.2026 выполнен на 22 одинаковых товарах. Для Python приведены медианы трёх прогонов; для n8n - сохранённый успешный запуск. Трафик учитывает тела ответов. Память n8n включает контейнер и рендерер.

Метрика	Playwright	AJAX requests	n8n
Записей в сравнении	22	22	22
Время прогона, с	2.9429	1.6528	3.385
Сетевых запросов	14	2	15
Полученные тела ответов, МБ	1.067404	0.026241	1.091501
Пиковая память, МБ	497.099	112.603	1570.055
Секунд на запись	0.1338	0.0751	0.1539
Устойчивость к редизайну, 1-5	3	4	3
Воспроизводимость	Python + requirements	Python + requirements	Экспорт + локальный API

Блокировка изображений, шрифтов и медиа сократила время на 33,39%, трафик на 73,59%. Массовое извлечение заняло 0,0118 с против 4,7922 с поэлементного - примерно в 406 раз быстрее.

Один запуск n8n использует один JS-рендеринг; 100 запусков - 100 рендерингов. Локальная схема не требует платного сервиса, но использует ресурсы компьютера.

Вывод

Для разового исследования удобен Playwright. Для ежедневного сбора 10 000 записей подходит разрешённый API или AJAX. n8n удобен для небольшого сценария по расписанию. Для сайта с антибот-защитой требуется официальный доступ к данным.

Файлы и запуск

Код: collect.py, webscraper_source.py, diagnose.py и render_api.py. Анализ: lab4.ipynb. Результаты: CSV, журнал и results.json. Схема: nocode/workflow.json. Порядок установки и запуска приведён в README.md.

Источники

ScrapingCourse: Button Click

Web Scraper: More / Laptops

Playwright: локаторы